Performance Testing

Date	15 July 2026
Team ID	SWTID-2026-5757
Project Name	Credit card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Web Browser (Google Chrome) and Flask Development Server
Type of Testing	Functional Testing / Manual Testing
Target Module / API	Credit Card Approval Prediction (`/result` endpoint) and Applicant Input Form
Test Environment	Local System (Flask Development Server)
Test Date	*08-07-2026*

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits credit card application	1	10	Prediction is displayed successfully without errors.
2	Multiple users submit applications simultaneously	10	30	All requests are processed correctly with acceptable response time.
3	Continuous prediction requests	20	60	Application remains stable and returns predictions for all requests.
4	Invalid or incomplete input submission	5	20	System validates input and displays appropriate error messages without crashing.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	~1.2 seconds	Pass	Prediction generated quickly during local testing.
2	Response Time (Max)	< 5 seconds	~2.3 seconds	Pass	Maximum response time remained within the target.
3	Throughput (Req/sec)	5 Req/sec	~6 Req/sec	Pass	Sufficient for a single-user Flask application.
4	Error Rate	< 1%	0%	Pass	No errors observed during functional testing.
5	CPU Utilization	< 80%	~35%	Pass	CPU usage remained low during testing.
6	Memory Utilization	< 80%	~42%	Pass	Memory consumption was within acceptable limits.

Step 4: Observations & Analysis

Key Findings

- The Random Forest model successfully predicted credit card approval with an overall accuracy of **86.40%**.
- Average prediction response time was less than **2 seconds** during local testing.
- The application processed user inputs correctly and generated prediction results without runtime errors.
- The system remained stable throughout functional testing.

Bottlenecks Identified

- Prediction accuracy is slightly affected by the **imbalanced dataset** (fewer approved/rejected samples in one class).
- The Flask application currently supports **single-user/local deployment** and is not optimized for heavy concurrent traffic.
- Prediction performance depends on the quality and completeness of the applicant's input data.

Optimization Steps Taken

- Applied **data preprocessing** by handling missing values and encoding categorical features.
- Removed unnecessary features such as **family_dependency** to simplify the model.

- Tuned the **Random Forest** hyperparameters (n_estimators, max_depth, min_samples_split, min_samples_leaf, and class_weight) to improve prediction performance.
- Organized the Flask application into modular components and added exception handling for robust execution.
- Used **Bootstrap** to create a responsive and user-friendly interface.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

These are the screenshots from google chrome/render deployment link, as a part of functional testing:-